

Assignment-1

Matrix Multiplication and Transpose Operations

Effective usage of Shared Memory helps to improve the performance of CUDA programs. Matrix Multiplication ($\mathbf{A} \times \mathbf{B} = \mathbf{C}$) and Matrix Transpose are couple of common examples. Implementing these matrix operations using dynamic tile sizes provides a good opportunity to study the performance improvement due to usage of Shared Memory.

We describe five separate problems/experiments (1.1, 1.2, 2, 3 and 4) below:

1. Basic Version of Matrix Multiplication

Implement a basic version of 2D matrix multiplication using a CUDA kernel. Each thread in the kernel computes one element/entry in the resulting product matrix $\mathbf{C}$ by reading the respective row vector from matrix $\mathbf{A}$ and column vector from matrix $\mathbf{B}$. Measure the execution time of the kernel.

1.1. Kernel launch using 1D params

Configure the grid and thread block sizes (as 1D or scalar values in `kernel<<<M,N>>>`) of the Matrix Multiplication kernel. Vary the block dimensions and thread dimensions in the code and observe its effect on the runtime of the kernel. Populate the table below with the runtime numbers against the respective configuration (should contain values for at least eight rows).

Trial No	#Thread Blocks (M)	Block Size, i.e., #Threads in a block (N)	Kernel Execution time (microsecs)	Your insights on the execution time
1				
2				
8				

Plot (bar chart) the runtime values for different block and thread dimension pairs (data points), where the X-axis is pair (M, N) and the Y-axis is time. You should plot at least eight data points.

1.2. Kernel launch using 2D params

Repeat the same experiment, but use a 2D grid and block sizes, i.e using CUDA *dim3* data type.

Trial No	2D Grid Size (X1,Y1)	2D Block Size (X2,Y2)	Kernel Execution time	Your insights on the execution time
1				
2				
8				

1.3. Sample o/p

Example:

```
./matmul_1d <M> <N> <path to matrix_a.csv> <path to matrix_b.csv>
```

```
Product Matrix of size 123 stored as matrix_c.csv
```

```
Kernel execution time: 456 microseconds
```

```
./matmul_2d <X1> <Y1> <X2> <Y2> <path to matrix_a.csv> <path to matrix_b.csv>
```

```
Product Matrix of size 123 stored as matrix_c.csv
```

```
Kernel execution time: 456 microseconds
```

output_{prob#}_{YOUR_ROLL_NUMBER}.txt should look like this:

```
Line1: 123
```

```
Line2: 456
```

2. Tiled Version of Matrix Multiplication

Modify the previous (basic version) kernel to use Shared Memory for fetching the operands from Global Memory. The tile sizes should be dynamically configurable (not `#defines`) via command-line arguments. Increase the tile size in steps till the maximum possible value and observe the corresponding change in execution time. You can tune and choose optimal kernel launch parameters (block sizes) and assign them in the code. Populate the table below with the runtime numbers against the respective configuration (should contain values for at least eight rows).

SI No	Tile Size	#Global Memory Access	#Shared Memory Access	Kernel Execution time	Your insights on the execution time
1	No Tiling (basic version)				
2					
8	Try using the entire Shared Memory Available				

Note: The logic to be used to compute memory access is outlined in the Appendix. We are interested in access to the elements of the input/output matrices only (and not other private variables that you might have used).

Plot the runtime for different tile sizes and report the optimal tile size. The X-axis will be the tile width and Y-axis is the execution time (us).

2.1. Sample o/p

```
./matmul_tiled <TILE_WIDTH> <path to matrix_a.csv> <path to
matrix_b.csv>
Product Matrix of size 123 stored as matrix_c.csv
Kernel execution time: 456 microseconds
```

3. Basic Version of Matrix Transpose

Implement a basic version of 2D matrix transpose using a CUDA kernel. Each thread in the kernel populates one element in the column vector of the resultant transpose matrix $\mathbf{A}^T$ by reading the respective element from the row vector of matrix $\mathbf{A}$. Measure the execution time of the kernel.

Configure the grid and thread block sizes (as 1D or scalar values in `kernel<<<M,N>>>`) of the matrix transpose kernel. Vary the block dimensions and thread dimensions in the code and observe its effect on the runtime of the kernel. Populate the table below with the runtime numbers against the respective configuration (should contain values for at least eight rows).

Trial No	#Thread Blocks (M)	Block Size i.e. #Threads in a block (N)	Kernel Execution time (microsecs)	Your insights on the execution time
1				
2				
8				

Plot (bar chart) the runtime values for different block and thread dimension pairs (data points), where the X-axis is pair (M, N) and the Y-axis is time. You should plot at least eight data points.

Based on this study, share your observations (with proper tables, figures wherever required) of the matrix transpose operation with respect to matrix multiplication.

3.1. Sample o/p

```
./mattrans_basic <M> <N> <path to matrix_a.csv>
Transpose Matrix of size 123 stored as matrix_c.csv
Kernel execution time: 456 microseconds
```

4. Tiled Version of Matrix Transpose

Modify the basic matrix transpose version kernel to use tiling. The tile sizes should be dynamically configurable (not `#defines`) via command-line arguments. Increase the tile size in steps till the maximum possible value and observe the corresponding change in execution time. You can tune and choose optimal kernel launch parameters (block sizes) and assign them in the code. Populate the table below with the runtime numbers against the respective configuration (should contain values for at least eight rows).

SI No	Tile Size	Kernel Execution time	Your insights on the execution time
1	No Tiling (basic version)		
2			
8	Try using the entire Shared Memory		

SI No	Tile Size	Kernel Execution time	Your insights on the execution time
	Available		

Plot the runtime for different tile sizes and report the optimal tile size. The X-axis will be the tile width and Y-axis is the execution time (us).

Did you use shared memory for this implementation? Share your observations (with proper numbers, tables and figures wherever required) with and without using shared memory.

Do you observe a difference in the impact of tiling optimization on the transpose of smaller (say ≤ 128) and larger matrices (say ≥ 2048)? You can generate these matrices on your own and perform the experiments.

4.1. Sample o/p

```
./mattrans_tiled <TILE_WIDTH> <path to matrix_a.csv>
Transpose Matrix of size 123 stored as matrix_a_trans.csv
Kernel execution time: 456 microseconds
```

5. Deliverables

Source files: Implement your program using CUDA C. Provide the .cu file(s) and a build.sh. The build.sh should generate all the required executables. So, running ./build.sh should generate: matmul_1d, matmul_2d, matmul_tiled, mattrans_basic and mattrans_tiled in one shot. Ensure the programs accept the command-line arguments exactly as shown in the respective sample o/p's under each problem.

Input files: Input matrices can be assumed to be square with a max size of 10K containing values of *int32* data type. Sample CSV files for the matrices are provided for reference. We will test your application with private input files.

Output files/values: The program should store the computed product matrix in CSV format to a file. It should also report the execution time of the kernel in microseconds. The execution time and the result matrix size should be stored in a file named output_{prob#}_{YOUR_ROLL_NUMBER}.txt

Report: Capture the findings in a table/plots/insights as indicated under each problem within a single report PDF file. Be descriptive and focus more on highlighting your findings/inferences.

Directory Structure

Assignment_folder (Name: {YOUR_ROLL_NUMBER_IN_CAPS}.zip)

```
|
|— main_<prob#>.cu          # Main assignment program files (multiple)
|— report.pdf              # Assignment report (single)
|— build.sh                # build commands(single)
|
|— public_test_cases        # Folder containing public test cases and o/ps
  |— {input_file_name}.csv  # Matrices(multiple)
  |— output_{prob#}_{YOUR_ROLL_NUMBER}.csv
  |— output_{prob#}_{YOUR_ROLL_NUMBER}.txt
```

Important Guidelines:

1. The **zip file name** must be your roll number in **UPPERCASE** (e.g., **CS21BTECH11001.zip**).
2. The **main program file** must be named exactly: main_1_1.cu, main_1_2.cu, main_2.cu, main_3.cu and main_4.cu respectively corresponding to problems 1.1, 1.2, 2, 3 and 4. You can create any supplemental source files, if you want.
3. The **public test cases folder** must be named **public_test_cases**. We have provided **matrix_a.csv** and **matrix_b.csv** as a reference for your testing. TAs will build/execute your program and run it with private test cases.
4. For each problem, the *corresponding output file must be named: output_{prob#}_{your roll number in UPPERCASE}.csv*
 - a. Example (matmul):
 - Input: **matrix_a.csv, matrix_b.csv**
 - Output: **output_1_1_CS21BTECH11001.csv**
 - b. Example (mattrans):
 - Input: **matrix_a.csv**
 - Output: **output_3_CS21BTECH11001.csv**

Note:

- The directory structure **must** match exactly as shown.
- Any deviation in folder structure or file naming will result in **automatic rejection** by the autograder.
- Strict adherence to naming conventions (mind the **spaces** also) is required; otherwise, your submission will **not be graded**.

Appendix

A) Measuring kernel execution time

Code snippet/logic used to measure kernel execution time

```
cudaEventRecord(start);
MyKernel<<<gridDim, blockDim>>>();
cudaEventRecord(stop);
cudaEventSynchronize(stop);
float ms = 0.0f;
cudaEventElapsedTime(&ms, start, stop);
printf("Kernel elapsed time: %f us\n", ms*1000);
```

B) Calculating # of Memory Accesses

Every read or write of a uint32 from/to memory is counted as one access.

For example, for the below code snippet, we calculate the:

#Global Memory Access as 64 and

#Shared Memory Access as 64.

```
__global__ void copyToShared(int *d_out, const int *d_in)
{
    __shared__ int tile[32];
    int tid = threadIdx.x;
    tile[tid] = d_in[tid]; // 32 Global reads, 32 SHM
writes
    d_out[tid] = tile[tid]; // 32 SHM reads, 32 Global
writes
}

copyToShared<<<1, 32>>>>(d_out, d_in);
```